

State space neural network with nonlinear physics for mechanical system modeling

Reese Eischens^a, Tao Li^b, Gregory W. Vogl^c, Yi Cai^{b,d}, Yongzhi Qu^{e,*} 

^a University of Minnesota Duluth, Duluth, MN, 55804, USA

^b The Hong Kong University of Science and Technology (Guangzhou), Smart Manufacturing Thrust, Nansha, Guangzhou 511400, China

^c National Institute of Standards and Technology, Gaithersburg, MD 20899, USA

^d The Hong Kong University of Science and Technology, Hong Kong

^e The University of Utah, Salt Lake City, UT, 84112 USA

ARTICLE INFO

Keywords:

Neural networks
Differential equations
Digital twin
Dynamic systems

ABSTRACT

Dynamic modeling of mechanical systems is important for the monitoring, diagnostics, control, and prediction of system behaviors. Modeling dynamic systems is one of the emerging tasks in scientific machine learning. Neural networks have been used to learn surrogate models for the underlying dynamics in the form of data-driven neural ordinary differential equations (NODEs). While most dynamical mechanical systems have some degree of nonlinearity within their dynamics, neural networks have shown potential in approximating dynamic systems with nonlinearities. However, despite the universal approximation capability of neural networks, this paper argues that by adding physics-aware nonlinear functions to the neural network model, the modeling accuracy of the neural network can be increased. In this paper, the construction of the nonlinear continuous-time state-space neural network (NLCSNN) is presented. The proposed approach can be used as a data-driven method for digital twin construction for monitoring, prediction, and reliability assessment. The NLCSNN improves upon the previously established continuous-time state-space neural network by increasing sensitivity to nonlinearity. The proposed NLCSNN is trained and validated using numerical and experimental examples, with results compared against those from several existing methodologies. Validation results show that the NLCSNN model can learn complex engineering dynamics without explicit knowledge of the underlying system. The modeling performance of the proposed data-driven approach outperforms a purely physics-based model, with results comparable to hybrid models. Additionally, the NLCSNN model achieved higher accuracy than the continuous-time state-space neural network (CSNN) model.

1. Introduction

Reliability analysis often requires the simulation of system dynamics. Xu and Spence studied the collapse reliability of reinforced concrete structures under wind excitation by analyzing the nonlinear dynamic response of these structures [1]. Dynamic systems can typically be expressed mathematically with ordinary differential equations (ODEs) or partial differential equations (PDEs), which model the underlying governing physical phenomena, such as fluid dynamics, heat transfer, and mechanical interactions. Accurate modeling of engineered systems with dynamic characteristics is critical for ensuring system reliability and safety [2]. When modeling unknown systems, exact physics-based methodologies may not be feasible due to incomplete knowledge of

the underlying physics. In such cases, data-driven or hybrid methodologies can be employed to address the unknown aspects of the system. Modeling partially known systems remains an area of active research [3–5].

Many mechanical systems have failure modes that can lead to catastrophic consequences, putting human lives at risk. To mitigate these dangers, prognostic tools can be developed to detect potential failures through digital twins and virtual sensors. A digital twin is a virtual replica of the system of interest, enabling analysis of how the system responds to inputs and facilitating failure prediction when there is a significant discrepancy between the measured and estimated outputs. Virtual sensors, meanwhile, use data from one variable to accurately infer another, allowing for real-time estimation of variables that would

* Corresponding author.

E-mail address: yongzhi.qu@utah.edu (Y. Qu).

<https://doi.org/10.1016/j.ress.2025.110946>

Received 14 June 2024; Received in revised form 11 November 2024; Accepted 18 February 2025

Available online 19 February 2025

0951-8320/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

otherwise be challenging to collect directly. For these tools to be effective in system prognostics, both digital twins and virtual sensors must achieve high fidelity and accuracy. Data-driven and hybrid methods have been applied to create digital twins and virtual sensors for systems with unknown dynamics [6]. We will provide a brief review of existing methods for dynamic system modeling. Two categories of methods will be discussed: pure data-driven methods and physics-aware methods. It is worth noting that, by "physics-aware," we refer to any methods that utilize physics-based properties, such as the derivative relationships between variables over time, and are not limited to accurately descriptive physics.

1.1. Data-driven methods

Various data-driven methods have been employed for modeling unknown systems when the underlying physics is not available to construct a model. Neural networks have been employed to represent nonlinear dynamical systems [7,8].

Convolutional neural networks (CNNs) can model system dynamics but require data segments of specific sizes to predict time series [9]. However, traditional convolutional neural network-based methods are typically limited in their capability for time series prediction due to the requirement for data of specific lengths as inputs and outputs. This limitation prevents CNNs from being used for real-time prediction and from functioning as digital twins or virtual sensors. Recurrent neural networks (RNNs) and long short-term memory networks (LSTMs) have also been applied to model the responses of dynamical systems [10,11]. RNNs and LSTMs use hidden states to transfer information forward in time. Due to the discrete nature of state propagation in RNNs and LSTMs, both methods are sampling rate-dependent and require a uniform sampling rate.

DeepONet is a data-driven neural operator used to learn the dynamics of a system [12]. DeepONet can approximate any functional relationship between two spaces. However, it requires specifically sized data for a domain input to calculate the output, which reduces its effectiveness for real-time prediction. The Fourier neural operator (FNO) is another neural operator that uses the Fourier transform as kernel functions to approximate the corresponding relationship between inputs and outputs [13]. Both DeepONet and the FNO require large, exhaustive datasets to learn the dynamics of a system.

The major limitation of pure data-driven methods is that, as black box models, they often lack extrapolation capabilities and require a thorough dataset that covers a fine mesh over the learning domain of the input space.

1.2. Physics-aware methods

Physics-informed neural networks (PINNs) are deep neural networks that can solve nonlinear partial differential equations [14]. PINNs can also be applied to system identification when the governing equation is known in form but has unknown coefficients. PINNs have been widely used as a hybrid model for reliability analysis [15,16]. PINNs require strong physics input via fully known underlying PDEs with potential unknown parameters. Physics-informed CNN (PhyCNN) incorporates classical physics equations into the loss calculations of a convolutional neural network, enhancing the model's accuracy [17]. Bilinear neural network methods (BNNMs) have also been developed to obtain the analytical solution of partial differential equations [18,19]. As shown in Ref. [18], a neural network model can be constructed with customized activation functions to represent the p-gBKP equation, which can be solved analytically.

Sparse identification of nonlinear dynamics (SINDy) is a regression-based data-driven approach that uses a predefined library of nonlinear basis functions to learn the system dynamics [20]. SINDy constructs a linear combination of nonlinear functions from a library to approximate system dynamics. This enables SINDy to accurately represent various

nonlinear functions or combinations of functions using its library. Because SINDy operates linearly over an underlying nonlinear functional space, it can potentially recover the exact underlying differential equations. However, SINDy may struggle to learn accurate dynamics if the library of nonlinear functions is insufficiently comprehensive for the system.

State-space models are often used to describe the dynamic evolution of system states over time. They can be used for both the dynamic modeling of intrinsic system properties and the system's degradation over time [21–23]. By incorporating the integral operator with the neural network, we can constrain the neural network to a state-space model that represents the underlying dynamics. Instead of using neural networks to directly map inputs to outputs, alternative methodologies have been proposed to represent differential functions for dynamic modeling. One such methodology, neural ordinary differential equations (NODE), is used to more accurately represent ordinary differential equations [24]. NODE-based methodologies, such as augmented neural ordinary differential equations (ANODE) [25] and second-order neural ordinary differential equations (SONODE) [26], use neural networks to represent a system's dynamics as a set of linear ordinary differential equations. These methods then apply numerical integration to approximate the integration needed to solve the ODEs between discrete time samples. This integration allows NODE to handle nonuniformly sampled data. Recently, NODE has also been applied to reliability analysis under accelerated fatigue testing [27]. As data-driven methods, these approaches accurately model unforced dynamical systems but struggle to model forced dynamical systems that do not exhibit a unique trajectory. Physics-informed NODEs have been proposed to address system identification problems when much of the underlying physics is known [28]. In a physics-informed formulation, however, the neural network plays an auxiliary role, primarily fitting the residual errors that the physics-based model may not capture. ODE-RNN is another continuous-time methodology, being an RNN with time-dependent states that evolve according to a NODE [29]. ODE-RNN can map sparsely sampled data more accurately. However, for regularly sampled data with small timesteps, it offers only a minimal accuracy improvement over an RNN, with a substantial increase in training time.

A recent neural network-based methodology that incorporates physics-aware principles is the continuous-time state-space neural network (CSNN). CSNN is a model based on the state-space modeling technique from classical system modeling and NODE [21,30,31]. Previous state-space methods were proposed but operated only with uniform discrete timesteps [32]. With the introduction of NODE, discrete state-space methods were expanded to operate in continuous time using numerical integration. The state-space modeling method can represent nonlinear systems if the nonlinearities are known. While PhyCNN [17] uses future timesteps and convolutional layers to model the system, CSNN uses previously measured timesteps to model the system. By following the state-space modeling technique, CSNN can achieve higher accuracy than PhyCNN and enables real-time operation. Therefore, CSNN provide a good framework to directly approximate and learn nonlinear engineering dynamics.

Admittedly, neural networks contain nonlinearities through activation functions that can approximate universal nonlinear functions. However, for many engineered systems, certain nonlinear terms are partially known and can be incorporated to enhance the model's interpolation, extrapolation, and explainability across a wide range of applications. Nonlinear dynamics are crucial when assessing system reliability [1,33,34]. Engineering dynamics often involve low-order polynomial terms as in low-order systems [35,36]. Neural networks have difficulties to fit polynomial functions [37] and do not extrapolate well, but they benefit from using bounded or piecewise linear equations to avoid explosion. However, this approach also limits the extrapolatability of neural networks. Combining the expressive power of polynomials with classic activation functions (such as sigmoid, ReLU, tanh) of neural networks is an emerging area of research that the machine

learning community has begun to explore [38,39]. This paper proposes a simple yet effective way to add explicit nonlinearities to neural network to address the emerging needs.

We argue that incorporating candidate nonlinearity functions into the state-space functions would significantly improve the expressivity of the neural network for modeling engineering dynamics. This paper outlines a methodology called the Nonlinear Continuous State Space Neural Network (NLCSNN) for dynamic system modeling. This approach can be applied to learn a model for a general, unknown, forced/unforced nonlinear dynamical system. The methodology builds upon NODE, with the addition of a physics-aware nonlinear layer to account for greater nonlinearities. The proposed method is expected to increase model accuracy and improve the extrapolatability.

The remainder of this paper is organized as follows: Section 2 outlines the proposed methodology, Section 3 presents the validation of model performance through numerical and experimental examples, and Section 4 provides the conclusions.

2. Revisit of node

A traditional NODE model is an effective approach for solving initial value problems. The NODE solves for the differential relationship between a state (x) and its temporal derivative ($\dot{x}$) by optimizing the neural network (f) in the equation, followed by numerical integration:

$$\dot{x}(t) = f(x(t)) \quad (1)$$

$$x[k+1] = x[k] + \text{ODESOLVE}(f(x(t))) \quad (2)$$

To solve for the state value at future timesteps beyond the initial conditions, numerical integration of the neural network is necessary. When a system is nonlinear, NODE can struggle to accurately model the nonlinear dynamics. NODEs for nonlinear systems take significantly longer to converge to a solution compared to linear systems. For certain nonlinear systems, a NODE may not converge at all.

A linear spring-mass-damper system is an example of an unforced linear oscillator, Fig. 1. The equation of motion for this spring-mass-damper system is given by:

$$m\ddot{x}(t) + c\dot{x}(t) + kx(t) = 0 \quad (3)$$

Where m is the mass, c is the damping coefficient, and k is the spring constant. The spring-mass-damper system was simulated using coefficient values of $m = 1$ kg, $c = 1$ N·s/m, and $k = 20$ N/m and initial conditions of $x(0) = 10$ m and $\dot{x}(0) = 0$ m/s.

A learning example with NODE is shown in Fig. 2, where the NODE model converges relatively well for both training and prediction.

A Duffing oscillator system is an example of a nonlinear oscillator. Physical systems that can be described as Duffing oscillators include pendulums, isolators, beams, and nonlinear electronic circuits [40]. The

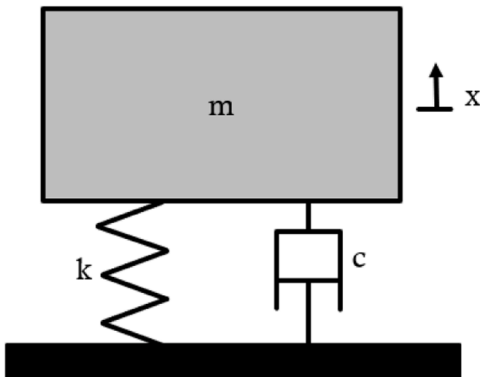


Fig. 1. Spring-mass-damper system.

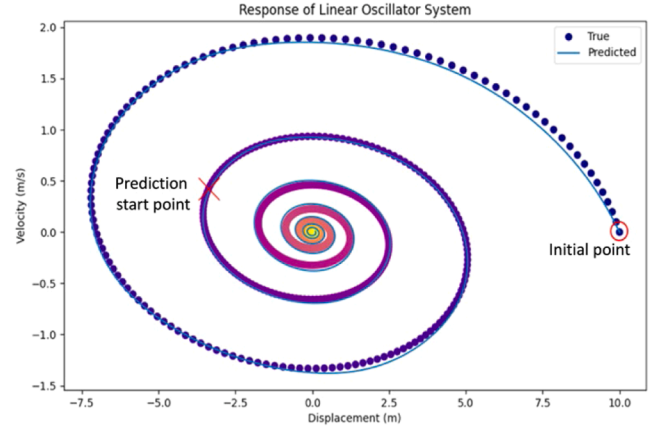


Fig. 2. NODE trained for linear oscillator system.

Duffing oscillator can exhibit chaotic behavior, which is challenging to model. It includes a nonlinear displacement term that leads to nonlinear dynamics. The equation of motion for the Duffing oscillator is given by:

$$m\ddot{x}(t) + c\dot{x}(t) + k_L x(t) + k_{NL} x^3(t) = 0 \quad (4)$$

where m is the mass, c is the damping coefficient, k_L is the linear spring constant and k_{NL} is the nonlinear spring constant. The Duffing oscillator was simulated using coefficient values of $m = 1$ kg, $c = 1$ N·s/m, $k_L = 20$ N/m, and $k_{NL} = 200$ N/m³ and initial conditions of $x(0) = 10$ m and $\dot{x}(0) = 0$ m/s. It can be seen from Fig. 3 that the NODE cannot converge fully to the nonlinear system during the training state and the prediction in a longer time horizon is inaccurate after a few cycles.

Additionally, training the NODE for the Duffing oscillator required approximately four times the number of epochs. Specifically, for the same NODE model, learning the linear oscillator took 11,000 epochs, while learning the Duffing oscillator took 41,000 epochs. This demonstrates that any additional nonlinearity in the differential equations leads to significantly longer training times, even for a simple, noise-free dataset. This observation motivates further research into more robust dynamic system modeling methods, especially for natural or engineered nonlinear systems.

3. Methodology

3.1. Background: state space modeling

A state-space representation models a dynamical system by converting the underlying governing differential equation into a series of first-order differential equations. These equations relate the system's

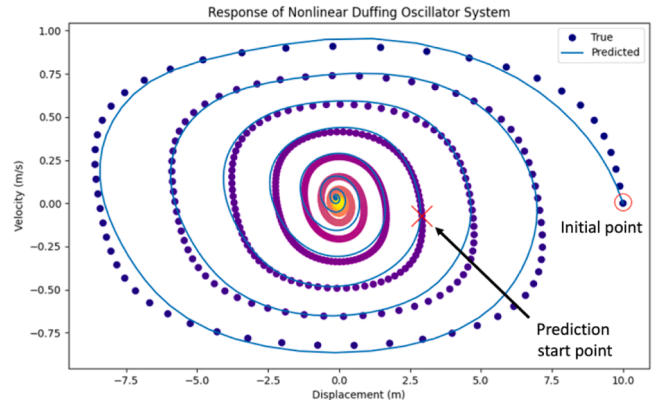


Fig. 3. NODE trained on the nonlinear Duffing oscillator system.

inputs and states to its output and state derivative. The state-space representation of a system uses state equations and observation equations to describe the dynamics:

$$\dot{x}(t) = A(t)x(t) + B(t)u(t) \quad (5)$$

$$y(t) = C(t)x(t) + D(t)u(t) \quad (6)$$

where A is the state matrix, B is the input matrix, C is the output matrix, D is the feedthrough matrix, x is the state vector, u is the input vector, and y is the output vector.

A state-space representation provides a rigorous foundation for system modeling. The matrices (A , B , C , and D) can be neuromorphically transformed into a neural network, where each network can be optimized to represent the system's dynamics. In a linear system, each matrix contains only constants. Neural networks can directly represent matrix multiplication by constructing a simple, single-layer network with no nonlinear activation function. This forms the basis for state-space neural networks (SSNNs). SSNNs model a system's dynamics by approximating the discrete state-space equations as neural networks [32]. The discrete state space equations can be rewritten as:

$$\dot{x}(t) = A(t)x(t) + B(t)u(t) \quad (7)$$

$$y(t) = C(t)x(t) + D(t)u(t) \quad (8)$$

A SSNN transforms the equations into:

$$x[k+1] = F(x[k], u[k]) \quad (9)$$

$$y[k] = G(x[k], u[k]) \quad (10)$$

where F and G are neural networks. The state space neural network is a physics-aware model that can model nonlinear dynamics due to the universal approximation theory of neural networks. To represent nonlinear systems, the neural networks F and G must contain nonlinear activation functions in at least one hidden layer within the SSNN model. However, the SSNN model can only model in discrete time and struggles with low or non-uniform sampling rates.

To model in continuous time and minimize errors from data with low and non-uniform sampling rates, the continuous-time state-space neural network was proposed recently [30]. The continuous-time state-space neural network represents the state-space equations in continuous time rather than discrete time, using the numerical integration method introduced by NODE. The neural networks in the CSNN are represented as:

$$\dot{x}(t) = NN_x(u, x, w) \quad (11)$$

$$y(t) = NN_y(u, x, w) \quad (12)$$

The state calculator (NN_x) calculates the state derivative. The output calculator (NN_y) calculates the output. In between timesteps the state calculator is numerically integrated, e.g., by using the Runge-Kutta method (RK4), to find the state at the next discrete time step. By integrating the state equation, the CSNN can be represented in the following equations and is shown in Fig. 4:

$$x(t + \Delta t) = x(t) + \int_t^{t+\Delta t} NN_x(u, x, w) dt \quad (13)$$

$$y(t) = NN_y(u, x, w) \quad (14)$$

In a traditional state-space model, the states (x) are assigned a physical meaning, such as position or velocity. However, the CSNN does not assign any physical meaning to the 'hidden' states within the model. To represent mechanical systems more accurately, the neural networks within the CSNN can be slightly modified. The CSNN may exclude all biases within the layers of the state and output calculators [30]. These

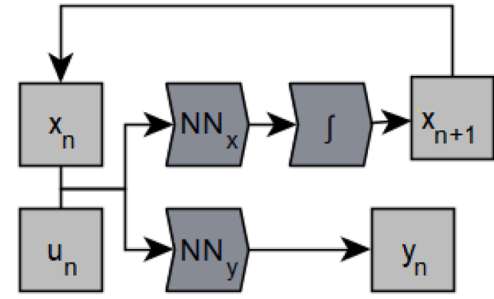


Fig. 4. CSNN architecture for the propagation of the hidden state (x) and output (y) using the input (u).

biases are removed to represent a damped system without drift. Additionally, the CSNN includes a linear layer in parallel with the nonlinear layers, which separates the nonlinear and linear components of the dynamics, thereby enhancing the model's accuracy [30]. This separation of nonlinear and linear dynamics allows the nonlinear layers to disregard the linear relationship. Removing the linear relationship reduces the complexity that the nonlinear layers need to learn, thereby improving overall performance.

3.2. Proposed nonlinear state space modeling

To account for nonlinearities, previous state-space neural networks have relied entirely on nonlinear neural networks. Traditionally, nonlinearities in neural networks are introduced through activation functions such as Sigmoid, ReLU, Tanh, etc. However, these nonlinearities operate in a surrogate manner and fail to capture the true physics-aware trends of the underlying dynamics. In many real-world engineering dynamics, low-order polynomial terms are often present and have proven effective in modeling nonlinear dynamic systems. For example, the Navier-Stokes equations include squared velocity components within the differential equations [41]. Another nonlinear system, the Duffing oscillator, can be used to model microelectromechanical systems that are used in some processors for clocking [42]. It is proposed that, in addition to the neural network, a physics-aware nonlinear layer should be added to explicitly model the nonlinearities. This added nonlinear layer is inspired by SINDy, functioning as a sparse layer capable of discovering nonlinear physics, which enhances the explainability and extrapolatability of the data-driven model.

Specifically, the proposed NLCSNN enhances the awareness of nonlinearities within the dynamical system by adding a nonlinear function library (NFL). It is expected that the additional nonlinearities will improve extrapolation and prediction outside the training domain by learning a weakly constrained model with candidate physics rules. The nonlinear function library is constructed from a small selection of candidate nonlinear functions, each with unknown but learnable coefficients. The proposed methodology can be considered a SINDy model embedded within the state-space neural network.

By concatenating the nonlinear function library to the inputs of a CSNN, an NLCSNN can be built and represented in equation form as:

$$x(t + \Delta t) = x(t) + \int_t^{t+\Delta t} NN_x(u, x, NFL(x, u), w) dt \quad (15)$$

$$y(t) = NN_y(u, x, NFL(x, u), w) \quad (16)$$

The nonlinear function library theoretically does not need to be exhaustive. Since the neural network is constructed from a combination of a linear layer and a nonlinear layer, the nonlinear functions in the library will be directly represented through the parallel linear layer. The parallel nonlinear layers will capture any remaining nonlinear functions that are not included in the function library. The larger the nonlinear

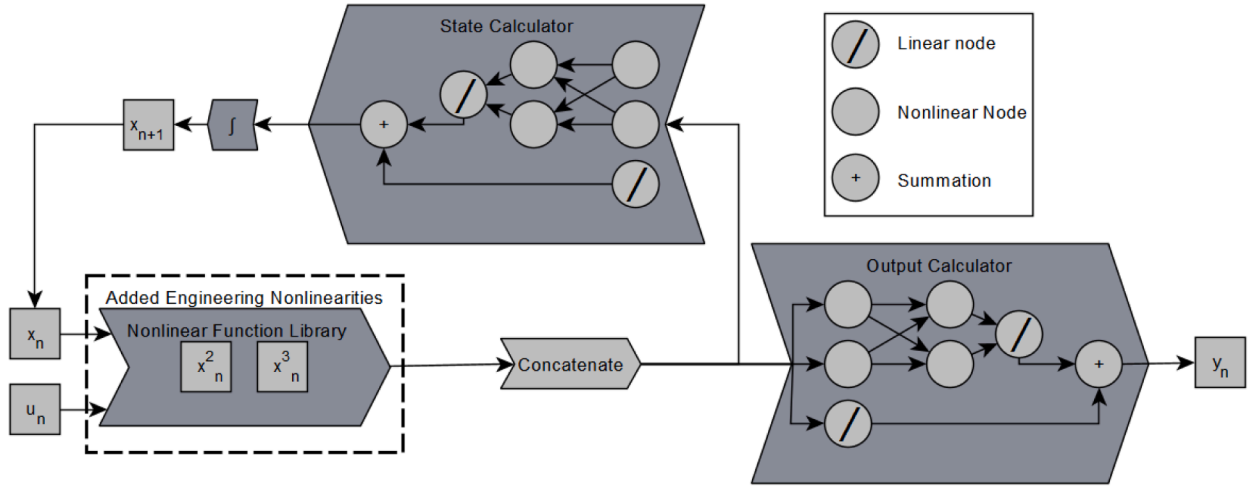


Fig. 5. NLCSNN architecture that uses current state (x_n) and input (u_n) to predict the output (y_n) and the next state (x_{n+1}).

function library, the more nonlinearities can be directly accounted for in the NLCSNN through the linear layer. Note that the library needs to be evaluated for each timestep, which will slightly increase the training time for each epoch. However, despite the increase in training time per epoch, a small library of likely nonlinear functions could reduce the total training time, as fewer epochs would be necessary to train the model.

The diagram of the learning structure is outlined in Fig. 5:

The corresponding training algorithm is shown in Algorithm 1.

4. Results & discussion

4.1. Case study 1 – Virtual excitation of a nonlinear Duffing oscillator using recorded seismic accelerations

In reliability and safety assessment, it is often required to analyze the structure response under seismic excitation [44]. In such cases, a dynamic model of the structure will be built first to model the underlying system. A nonlinear Duffing oscillator system with a single degree of freedom is represented as [17]:

$$m\ddot{x}(t) + c\dot{x} + k_L x(t) + k_{NL} x^3(t) = m\ddot{u}(t) \quad (17)$$

where $x(t)$ is the system displacement, the system parameters are mass ($m = 1$ kg), damping coefficient ($c = 1$ N·s/m), linear spring constant ($k_L = 20$ N/m), and nonlinear spring constant ($k_{NL} = 200$ N/m³), and the input to the system ($\ddot{u}(t)$) is the measured seismic acceleration. The system is simulated for each recorded seismic acceleration signal and solved via numerical integration. The seismic acceleration dataset consists of 99 different excitations sampled at 20 Hz. The first 10 excitations

were used for training, while the remaining 89 excitations were used for testing. This disproportionate separation can be understood since the system is deterministic and time-invariant, and the data is mostly noise-free. In theory, the internal dynamics can be accurately learned from just one trajectory if that trajectory can represent the underlying vector field with intrinsic linear and nonlinear behaviors. During training, all data was normalized to have a maximum value of 1, and the predictions were appropriately scaled.

To evaluate the direct effects of adding the nonlinear function library to the CSNN, the sizes of the neural networks were constrained to be the same. The CSNN was created with a state dimension of two. The state calculator was designed with two nonlinear hidden layers, each containing two nodes, as well as a parallel linear layer with two nodes. The output calculator was created with the same dimensions as the state calculator. Three different NLCSNNs were developed, each with a different nonlinear function library using the cubic functions of the input, the state, and both the input and the state.

Since a cubic function is present within the nonlinear system dynamics, any NLCSNN with a cubic state in its library should directly enhance the model's accuracy. All NLCSNN models are expected to perform as well as or better than the CSNN. As shown in Fig. 6 and Table 1, the NLCSNN models with nonlinear states exhibited a drastic reduction in testing losses compared to the CSNN. It is worth noting that the CSNN has been shown to have significantly less error than the PhyCNN in previous work [30].

For general unknown system modeling, using nonlinearities in both the input ($u(t)$) and state ($x(t)$) is recommended. In contrast, as seen in Table 1, the NLCSNN that used only nonlinear states was more accurate than the NLCSNN that incorporated nonlinearities for both the input and the state. However, this observation was made for a simple simulated system that contained nonlinearities only in the state. In real-world systems, unknown nonlinearities could be present in both the state and input. Hence, with sufficient training, a model can disregard a function within the library if it is not present in the dynamics.

To compare the computational efficiency of different models in converging, the time to reach the highest testing loss was measured. Since the CSNN had the highest testing loss, all the NLCSNN models were trained once they achieved a loss below that of the CSNN model, even if the model became more accurate in the future. While the CSNN had the lowest training time per epoch, it required the highest number of epochs to achieve a trained model, resulting in the longest total training time, as shown in Table 2. The loss curves for all models are plotted in Fig. 7. The NLCSNN models with cubic states experienced a relative time increase of 26 % per training epoch but required a much lower number of epochs to train, resulting in about a 78 % and 50 % relative decrease

Algorithm 1

Model Training.

```

Split training data into a matrix with each row being one data segment over which to
train
for row in TrainingDataMatrix
  Set initial state:  $x_0 = 0$ 
   $n = 0$ 
  while  $n < \text{Length of TrainingDataMatrix}$ 
    Evaluate predicted output using output calculator:  $\hat{y}_n = \text{NNy}(x_n, u_n)$ 
    Evaluate state using numerical integration:  $x_{n+1} = \text{RK4}(\text{NNx}(x_n, u_n))$ 
    Increment the index to move forward in time:  $n = n + 1$ 
  end while
Calculate loss using the loss function via the root mean square (RMS):  $L = \text{RMS}(y - \hat{y})$ 
Update model parameters by using the loss function and Adam optimizer algorithm [43]
end for

```

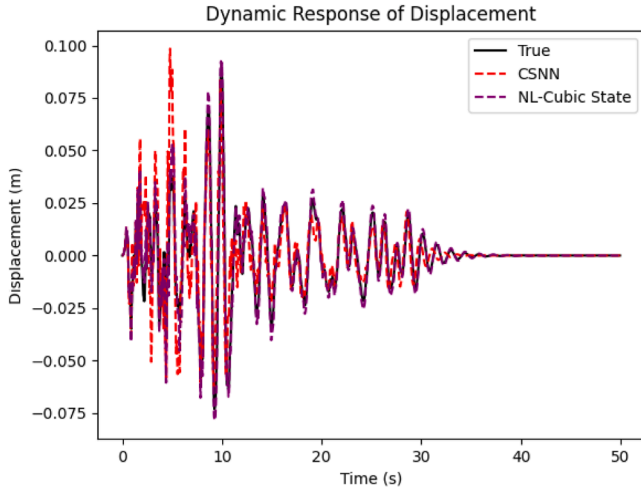


Fig. 6. The CSNN and NLCSNN with cubic states evaluated on a testing data segment for the Duffing oscillator system.

Table 1

Normalized testing losses for the CSNN and the NLCSNNs with different nonlinear function libraries.

	Normalized testing loss
CSNN	0.0451
NLCSNN (NL for state and input)	0.0040
NLCSNN (NL for input only)	0.0444
NLCSNN (add NL for state only)	0.0035

Table 2

Time for each methodology to achieve the same training loss.

	Average time per epoch (s)	Total time for training (min)
CSNN	18.71	87.00
NLCSNN (state and input)	23.41	67.50
NLCSNN (input only)	20.60	81.03
NLCSNN (state only)	23.83	43.30

in total training time. Not only did the NLCSNN models take less time to achieve accuracy comparable to the CSNN, but the NLCSNN models also exhibited higher accuracy.

4.2. Case study 2 - Continuous damping control damper

A continuous damping control (CDC) damper was excited with both harmonic and random displacement excitations. A CDC damper is a mechanical system that provides a resistive force opposite to velocity. Unlike a static damper, a current can be applied to the CDC damper to slightly alter its internal dynamics and, consequently, its resulting damping forces.

Fig. 8 shows a schematic of the CDC damper experimental setup as well as a picture of the test rig. To generate training data, a hydraulic actuator was used to excite the CDC damper using a harmonic displacement signal. For various excitations, the system displacement was measured while a current is applied to the CDC damper. Specifically, for the harmonic excitations, displacement amplitudes of 5 mm, 10 mm, 30 mm, and 50 mm were tested with frequencies of 0.41 Hz, 0.83 Hz, 1.66 Hz, 4.97 Hz, and 6.62 Hz. For 11 combinations of displacement amplitude and frequency, nine currents of 0 A, 0.2 A, 0.4 A, ..., 1.6 A were applied to the CDC damper to change the system dynamics. In each of the 99 tests, the system displacement and force were recorded. The velocity and acceleration are found by using the first- and second-order derivatives of the displacement signal, respectively. The

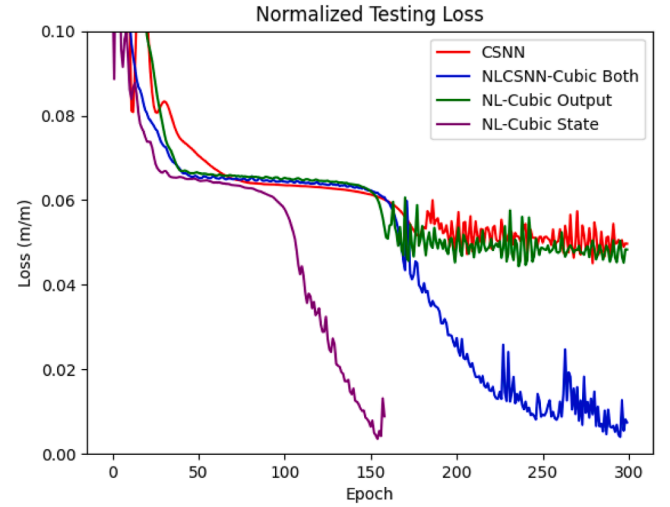


Fig. 7. Testing losses over epochs for the CSNN and NLCSNN models with different nonlinear function libraries.

displacement, velocity, and acceleration signals serve as the inputs to the CDC damper system models, while the force recorded on the fixed end of the CDC damper will serve as the model output.

For the testing data, a random excitation displacement was applied with various currents to the CDC damper. However, the “random” excitation is fixed; that is, the same random excitation was applied to the system for each current. Although the damper can adjust the current in real time, the current was kept constant during all excitations. The current levels for the random excitations were 0.4 A, 0.6 A, 0.8 A, and 1.2 A. The data was normalized to have a maximum value of 1 for training and testing, with the predictions appropriately scaled.

With a lumped mass model, the damping force of CDC dampers can be model with the following piecewise linear equations [45]:

$$f_c(\dot{x}, I) = \begin{cases} c_1(I) \dot{x} + [c_2(I) - c_1(I)]v_c(I) & \dot{x} < v_c(I) \\ c_2(I) \dot{x} & v_c(I) < \dot{x} < 0 \\ c_3(I) \dot{x} & 0 < \dot{x} < v_r(I) \\ c_4(I) \dot{x} + [c_3(I) - c_4(I)]v_r(I) & \dot{x} > v_r(I) \end{cases} \quad (18)$$

where I is the current and c_1 , c_2 , c_3 , and c_4 are slopes of the force-velocity curve of the CDC damper, and v_r and v_c are the turning velocities in the force-velocity curve. These parameters will change as the current I changes, and their relationships can be developed by fitting the measured force-velocity data. Two variables with the largest effect on the damping force within the CDC damper are the current and the velocity. The hybrid methodology presented in [45] uses the current and velocity to assemble a lumped parameter model based on a piecewise combination of four linear functions of current and velocity. Since the lumped parameter model cannot account for the nonlinear hysteresis that exists in the CDC damper (see Figs. 10 and 11), a neural network is added to the lumped parameter model to create a nonlinear hybrid method that increases the prediction accuracy.

To achieve the goal of developing fully data-driven methods with hypothetical physics to model the dynamics of the CDC damper, a CSNN and an NLCSNN were created with the same number of layers and nodes in each layer. Since the underlying equation of the system is unknown, the nonlinear state library consists of nonlinearities in the input and state, up to 3rd order polynomials. To model the coupling between velocity and displacement, the term $\dot{x} \cdot x$ was also included. The accuracies of the CSNN and the NLCSNN are compared to the accuracy of a deep neural network of similar size.

The CSNN and NLCSNN models are trained on the first 2.25 s (1152 samples) of the training data. Both models can extrapolate when given

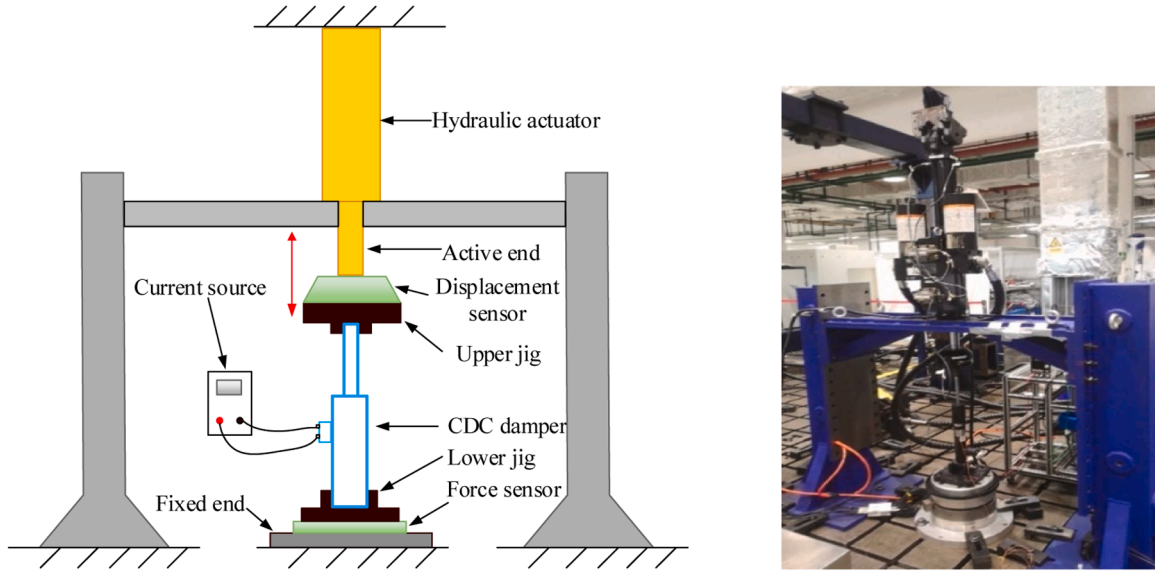


Fig. 8. Diagram of the CDC damper experimental setup and the actual test rig.

the inputs of displacement, velocity, and acceleration. This capability allows both models to be used in real-time to predict the output force of the CDC damper.

Fig. 9 shows the comparison between the lumped parameter model and the NLCSNN for the displacement-force relationship under different current settings, while Fig. 10 presents the corresponding comparison for the velocity-force relationship. The performance of the NLCSNN model is comparable to that of the lumped parameter model.

Fig. 11 shows the modeling performance comparison between the NLCSNN model, the lumped parameter model, and the hybrid model [45] with damper currents of 1.0 A and 1.4 A. The hybrid model performs the best, while the lumped parameter model fails to account for the nonlinear hysteresis that exists in the CDC damper. The hybrid method is based on the combination of a neural network and the lumped parameter model, which enables the addition of hysteresis to improve the overall performance. The proposed data-driven model can capture both the hysteresis and the nonlinearities in the physical system response.

Fig. 12 shows the time domain comparison of the predictions based on different models. It is seen that the pure data-driven model models the damper dynamics more accurately than the lumped parameter model.

Fig. 13 shows the CSNN and NLCSNN prediction results for random excitations. The CSNN generally models the CDC damper well. However,

the CSNN results in relatively small mismatches at the force peaks and troughs, while the NLCSNN demonstrates more robust predictions at these locations.

Even if the hybrid method is more accurate, it can only be constructed for systems with known analytical forms of the dynamics. The CDC damper could be just one nonlinear component of a larger nonlinear system, where the physics are largely unknown. Using hybrid methods to model these highly complex systems would become increasingly challenging. Therefore, hybrid methods are not feasible for systems with unknown dynamics.

To quantitatively evaluate different machine learning models, the root mean square error (RMSE) and the mean absolute error (MAE) were calculated for the NLCSNN, CSNN, and a deep neural network (DNN) of the same size. Tables 3 and 4 show the performance metrics for the three models calculated over 1270 s for the random excitation data collected at four current levels. Both the NLCSNN and CSNN considerably outperform the DNN, and the NLCSNN outperforms the CSNN except at the current level of 0.6 A. The standard deviations of the NLCSNN metrics are generally smaller than those for the CSNN due to the inclusion of the nonlinear term. With the best prediction performance, the NLCSNN is more robust than both the CSNN and the DNN.

The NLCSNN shows roughly a 15 % relative decrease in testing losses compared to the CSNN and approximately a 40 % decrease compared to the DNN for the entire testing dataset. In the first case study for the

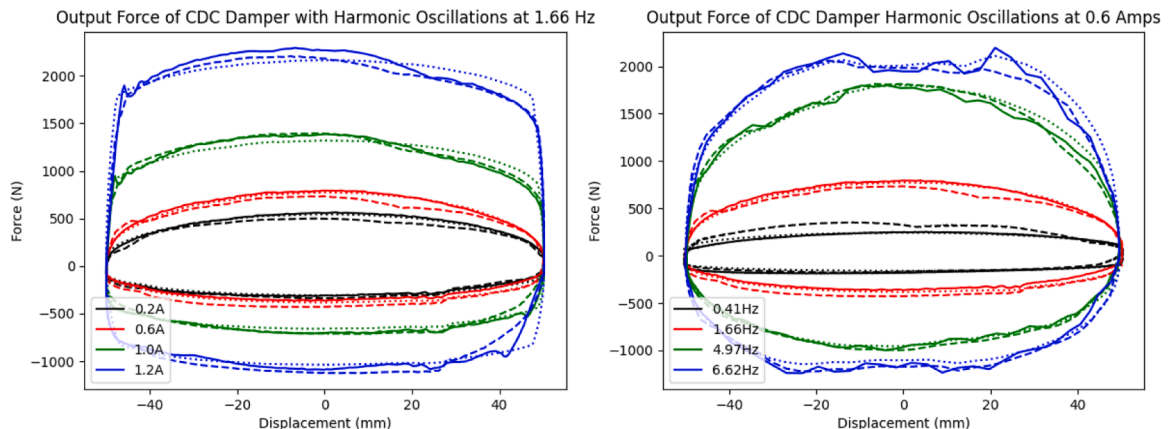


Fig. 9. Lumped parameter model [45] (dotted) and NLCSNN (dashed) compared to true output force (solid) vs displacement for the CDC damper.

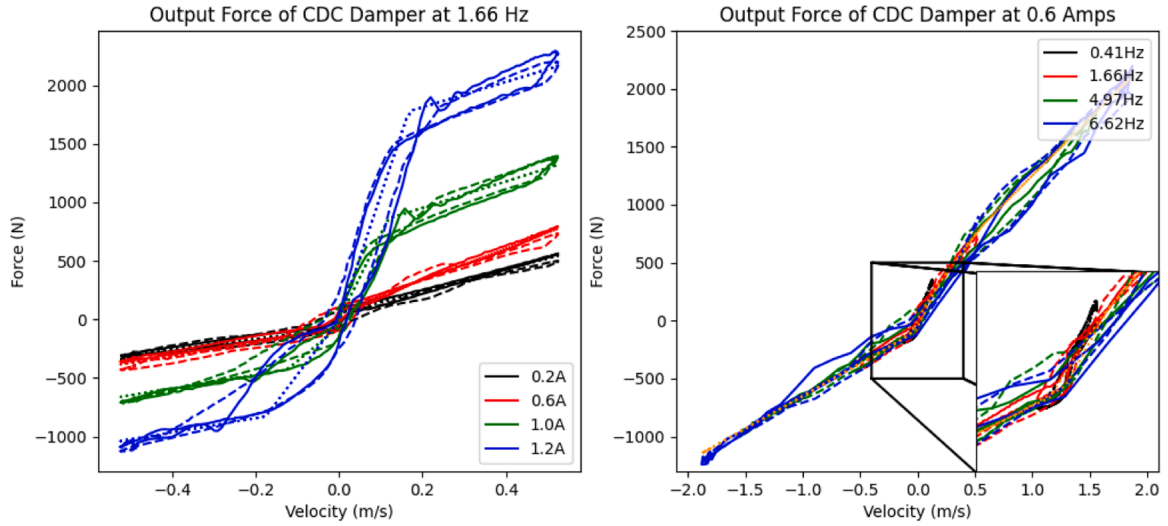


Fig. 10. Lumped parameter model [45] (dotted) and NLCSNN (dashed) compared to output force (solid) vs true velocity for the CDC damper.

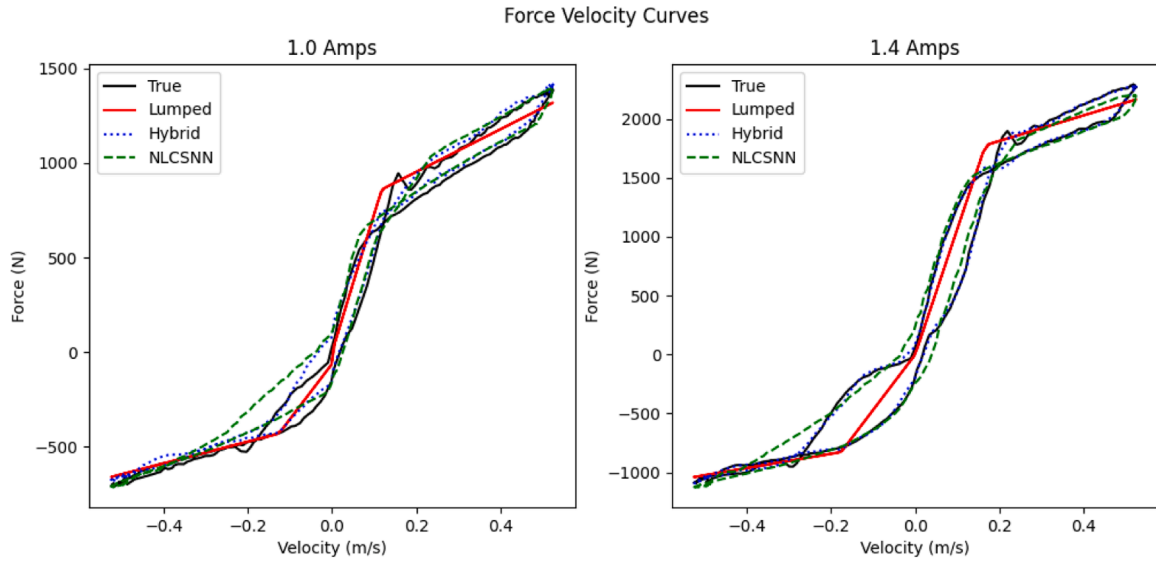


Fig. 11. Force velocity curves for different currents with the same external excitation of a 50 mm displacement wave with a frequency of 1.66 Hz.

Duffing oscillator, the NLCSNN performed considerably better (with the error being one order of magnitude smaller) than the CSNN. In the second case study, the NLCSNN does not demonstrate such a drastic performance increase. One reason for this may be that there is less nonlinearity present in the CDC damper than in the simulated Duffing oscillator, as seen in Fig. 10. Another possible reason for the performance difference of the NLCSNN in the two case studies is that the exact nonlinear term was included as part of the nonlinear function library in the first case study, while the exact nonlinearities are unknown in the second case study.

As shown in Table 5, both the CSNN and the NLCSNN exhibit longer training times than the DNN due to the added integration step. The NLCSNN required 30 % more training time than the CSNN, as it calculates a nonlinear function library at each time step and appends it to the input array. It is worth noting that the same number of training epochs is used for both the NLCSNN and the CSNN in this case study. However, in scenarios where the NLCSNN achieves faster convergence due to the added nonlinearities, a potential reduction in training time may be possible.

5. Conclusions

The NLCSNN was proposed by adding a modified SINDy layer to the continuous state-space neural network. With the addition of a nonlinear function library, the proposed model became more sensitive to nonlinear physics. Two examples were evaluated. The first case study involves a simulated nonlinear ODE, a Duffing oscillator, with seismic acceleration inputs. This example demonstrates that the NLCSNN offers the highest accuracy and lowest training times. The NLCSNN achieves higher accuracy by directly accounting for some of the nonlinearities in its nonlinear function library. Although the NLCSNN takes longer to train per epoch, it requires fewer total epochs than the traditional CSNN, resulting in shorter total training times. The second case study focuses on the CDC damper system, a real-world physical system that was excited using sinusoidal excitations of various amplitudes and frequencies. This example shows that the NLCSNN provides a performance boost not only for simulated systems with a high degree of nonlinearity but also for real systems with unknown nonlinearities. The performance boost comes with an overhead in training time with the same number of training epochs.

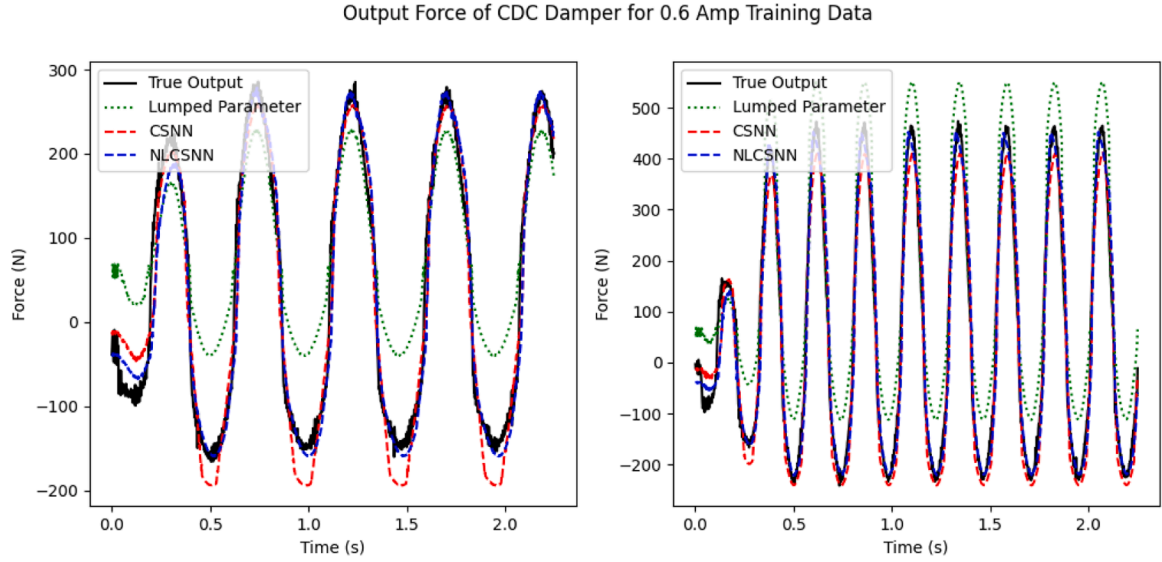


Fig. 12. Evaluation of the CSNN, NLCSNN, and lumped parameter models with an external sinusoidal excitation amplitude of 10 mm at frequencies of 2.07 Hz (left) and 4.14 Hz (right).

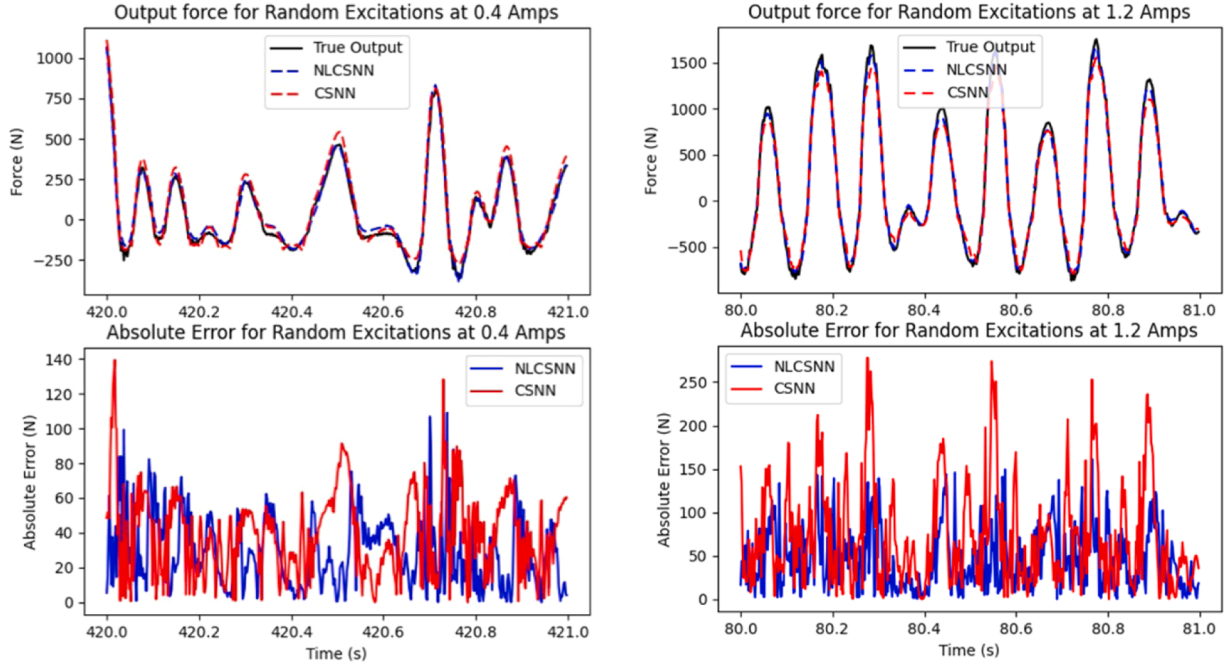


Fig. 13. CSNN and NLCSNN model outputs for the random excitation testing data.

Table 3

Mean absolute error (MAE) of the force predictions for the random excitation data segments. The metrics have the form of “mean $\pm$ standard deviation.”.

Current (A)	NLCSNN (N)	CSNN (N)	DNN (N)
0.4	32.0 $\pm$ 20.5	41.6 $\pm$ 23.2	59.9 $\pm$ 39.0
0.6	32.6 $\pm$ 30.4	36.8 $\pm$ 24.1	68.6 $\pm$ 51.9
1.0	40.8 $\pm$ 37.3	47.3 $\pm$ 41.4	81.0 $\pm$ 72.8
1.2	74.4 $\pm$ 67.1	80.2 $\pm$ 71.2	90.8 $\pm$ 85.7
Average	44.9	51.4	75.1

One limitation of the methods is that when the underlying nonlinearity is not included in the library, the improvement over neural networks may be limited. In future work, we will explore ways to incorporate generic nonlinearities and test the performance by adding

Table 4

Root mean square error (RMSE) of the force predictions for the random excitation data segments. The metrics have the form of “mean $\pm$ standard deviation.”.

Current (A)	NLCSNN (N)	CSNN (N)	DNN (N)
0.4	38.0 $\pm$ 40.7	47.6 $\pm$ 49.8	71.5 $\pm$ 75.4
0.6	44.5 $\pm$ 60.4	43.9 $\pm$ 54.7	85.9 $\pm$ 104.2
1.0	55.23 $\pm$ 73.3	62.9 $\pm$ 93.3	108.9 $\pm$ 151.6
1.2	100.2 $\pm$ 137.2	107.2 $\pm$ 141.0	124.9 $\pm$ 166.3
Average	59.5	65.4	97.8

sparse regularization to the optimization process for learning unknown engineering dynamics. More specifically, when a large library is used, the ℓ_1 norm of the coefficients of the nonlinear library can be added to

Table 5

Training Time for each methodology.

	Average time per epoch (minutes)	Total time for training (hours)
DNN	1.4	2.33
CSNN	12.3	20.47
NLCSNN	16.1	26.75

the total loss function to enforce sparsity. Also, since the nonlinear library is concatenated with the neural network output from the previous layer, a sparse penalty in terms of ℓ_1 can be added to the output network to enforce sparsity by eliminating unnecessary nonlinearities. Additionally, designing a robust algorithm that can handle noisy measurements is another area for future investigation.

CRedit authorship contribution statement

Reese Eischens: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Tao Li:** Writing – review & editing, Visualization, Validation, Resources, Methodology, Investigation, Formal analysis, Data curation. **Gregory W. Vogl:** Writing – review & editing, Validation, Supervision, Methodology, Investigation, Formal analysis. **Yi Cai:** Writing – review & editing, Resources, Investigation, Formal analysis. **Yongzhi Qu:** Writing – review & editing, Writing – original draft, Validation, Methodology, Investigation, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

Reese Eischens and Yongzhi Qu acknowledge that this work was supported in part by the Department of Commerce of the United States under contract nos. 70NANB22H109 and 70NANB22H109T.

References

- [1] Xu L, Spence SM. Collapse reliability of wind-excited reinforced concrete structures by stratified sampling and nonlinear dynamic analysis. *Reliab Eng Syst Saf* 2024; 110244.
- [2] Kakhia S, et al. Inverse uncertainty quantification of a mechanical model of arterial tissue with surrogate modelling. *Reliab Eng Syst Saf* 2023;238:109393.
- [3] Rahman A, et al. Neural ordinary differential equations for nonlinear system identification. In: 2022 American Control Conference (ACC). IEEE; 2022.
- [4] Fu X, et al. Modeling unknown dynamical systems with hidden parameters. *J Mach Learn Model Comput* 2022;3(3).
- [5] De S, et al. Bi-fidelity modeling of uncertain and partially unknown systems using DeepONets. *Comput Mech* 2023;71(6):1251–67.
- [6] Zio E, Miqueles L. Digital Twins in safety analysis, risk assessment and emergency management. *Reliab Eng Syst Saf* 2024;110040.
- [7] Ogunmolu, O., et al., Nonlinear systems identification using deep dynamic neural networks. *arXiv preprint arXiv:1610.01439*, 2016.
- [8] Masri, S., A. Chassiakos, and T. Caughey, Identification of nonlinear dynamic systems using neural networks. 1993.
- [9] Wu R-T, Jahanshahi MR. Deep convolutional neural network for structural dynamic response estimation and system identification. *J Eng Mech* 2019;145(1): 04018125.
- [10] Gajamannage K, et al. Recurrent neural networks for dynamical systems: applications to ordinary differential equations, collective motion, and hydrological modeling. *Chaos: Interdiscip J Nonlinear Sci* 2023;(1):33.
- [11] Wang Y. A new concept using LSTM Neural Networks for dynamic system identification. In: 2017 American control conference (ACC). IEEE; 2017.
- [12] Lu L, et al. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nat Mach Intell* 2021;3(3):218–29.
- [13] Li, Z., et al., Fourier neural operator for parametric partial differential equations (2020). *arXiv preprint arXiv:2010.08895*, 2010.
- [14] Raissi M, Perdikaris P, Karniadakis GE. Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J Comput Phys* 2019;378:686–707.
- [15] Xu Y, et al. Physics-informed machine learning for reliability and systems safety applications: state of the art and challenges. *Reliab Eng Syst Saf* 2023;230:108900.
- [16] Miele S, Karve P, Mahadevan S. Multi-fidelity physics-informed machine learning for probabilistic damage diagnosis. *Reliab Eng Syst Saf* 2023;235:109243.
- [17] Zhang R, Liu Y, Sun H. Physics-guided convolutional neural network (PhyCNN) for data-driven seismic response modeling. *Eng Struct* 2020;215:110704.
- [18] Zhang R-F, Billge S. Bilinear neural network method to obtain the exact analytical solutions of nonlinear partial differential equations and its application to p-gBKP equation. *Nonlinear Dyn* 2019;95:3041–8.
- [19] Zhang R-F, et al. The interference wave and the bright and dark soliton for two integro-differential equation by using BNNM. *Nonlinear Dyn* 2023;111(9): 8637–46.
- [20] Brunton SL, Proctor JL, Kutz JN. Sparse identification of nonlinear dynamics with control (SINDyC). *IFAC-PapersOnLine* 2016;49(18):710–5.
- [21] Pulido B, et al. State space neural networks and model-decomposition methods for fault diagnosis of complex industrial systems. *Eng Appl Artif Intell* 2019;79:67–86.
- [22] Kumar A, et al. State-space modeling and novel entropy-based health indicator for dynamic degradation monitoring of rolling element bearing. *Reliab Eng Syst Saf* 2022;221:108356.
- [23] Liu J, et al. Box-cox transformation based state-space modeling as a unified prognostic framework for degradation linearization and RUL prediction enhancement. *Reliab Eng Syst Saf* 2024;244:109952.
- [24] Chen RT, et al. Neural ordinary differential equations. *Adv Neural Inf Process Syst* 2018:31.
- [25] Dupont E, Doucet A, Teh YW. Augmented neural odes. *Adv Neural Inf Process Syst* 2019:32.
- [26] Norcliffe A, et al. On second order behaviour in augmented neural odes. *Adv Neural Inf Process Syst* 2020;33:5911–21.
- [27] Khakifirooz M, et al. Neural ordinary differential equation for sequential optimal design of fatigue test under accelerated life test analysis. *Reliab Eng Syst Saf* 2023; 235:109242.
- [28] Lai Z, et al. Structural identification with physics-informed neural ordinary differential equations. *J Sound Vib* 2021;508:116196.
- [29] Rubanova Y, Chen RT, Duvenaud DK. Latent ordinary differential equations for irregularly-sampled time series. *Adv Neural Inf Process Syst* 2019:32.
- [30] Li H-W, et al. Modeling of forced-vibration systems using continuous-time state-space neural network. *Eng Struct* 2024;302:117329.
- [31] Rangapuram SS, et al. Deep state space models for time series forecasting. *Adv Neural Inf Process Syst* 2018:31.
- [32] Zamarréño JM, Vega P. State space neural network. Properties and application. *Neural Netw* 1998;11(6):1099–112.
- [33] Bakeer T. General partial safety factor theory for the assessment of the reliability of nonlinear structural systems. *Reliab Eng Syst Saf* 2023;234:109150.
- [34] Gao Z, et al. Nonlinear slow-varying dynamics-assisted temporal graph transformer network for remaining useful life prediction. *Reliab Eng Syst Saf* 2024;248:110162.
- [35] Paduart J, et al. Identification of nonlinear systems using polynomial nonlinear state space models. *Automatica* 2010;46(4):647–56.
- [36] Cooper SB, et al. Polynomial nonlinear state space identification of an aero-engine structure. *Comput Struct* 2020;238:106299.
- [37] Liu, Z., et al., Discovering new interpretable conservation laws as sparse invariants (2023). URL <http://arxiv.org/abs/2305.19525>.
- [38] Zhang, J., RPN: reconciled Polynomial network towards unifying PGMs, kernel SVMs, MLP and KAN. *arXiv preprint arXiv:2407.04819*, 2024.
- [39] Liu, Z., et al., Kan: Kolmogorov-arnold networks. *arXiv preprint arXiv:2404.19756*, 2024.
- [40] Brennan MJ, Kovacic I. Examples of physical systems described by the Duffing equation. *The Duffing equation: nonlinear oscillators and their behaviour*. 2011. p. 25–53.
- [41] Constantin P, Foias C. *Navier-stokes equations*. University of Chicago Press; 1988.
- [42] Tajaddodianfar F, Yazdi MRH, Pishkenari HN. Nonlinear dynamics of MEMS/ NEMS resonators: analytical solution by the homotopy analysis method. *Microsyst Technol* 2017;23:1913–26.
- [43] Kingma, D.P. and J. Ba, Adam: a method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [44] Li J-Y, Lu J, Zhou H. Reliability analysis of structures with inerter-based isolation layer under stochastic seismic excitations. *Reliab Eng Syst Saf* 2023;235:109222.
- [45] Li T, Liao Z, Cai Y. Physical modeling for digital twin of continuous Damping control damper. *J Manuf Process* 2023;99:96–104.